

3.5: Codes

Problem Statement: Implement K-Nearest Neighbors algorithm

```
import math
from collections import Counter

def euclidean_distance(p1, p2):
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(p1, p2)))

def knn_predict(train_data, test_point, k):
    """
    train_data: list of tuples -> ([features], label)
    test_point: list of features
    k: number of neighbors
    """

    # Compute distances
    distances = [
        (euclidean_distance(features, test_point), label)
        for features, label in train_data
    ]

    # Sort by distance
    distances.sort(key=lambda x: x[0])

    # Take first k labels
    k_nearest_labels = [label for _, label in distances[:k]]

    # Majority vote
    return Counter(k_nearest_labels).most_common(1)[0][0]

train_data = [
    ([1.0, 2.0], "A"),
    ([1.5, 1.8], "A"),
    ([5.0, 8.0], "B"),
    ([6.0, 9.0], "B"),
    ([1.2, 0.8], "A"),
    ([7.0, 10.0], "B"),
]

test_point = [4.0, 5.0]

prediction = knn_predict(train_data, test_point, k=3)

print("Prediction:", prediction)

Prediction: A
```